

```

CREATE TABLE IF NOT EXISTS lugares (
  id          BIGSERIAL PRIMARY KEY,
  nombre      TEXT      NOT NULL,
  direccion   TEXT,
  comuna      TEXT,
  region      TEXT,
  lat         NUMERIC(9,6),
  lon         NUMERIC(9,6),
  capacidad_maxima INTEGER,
  timezone    TEXT      NOT NULL DEFAULT 'America/Santiago',
  activo      BOOLEAN    NOT NULL DEFAULT TRUE,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX IF NOT EXISTS idx_lugares_activo ON lugares (activo);

```

```

CREATE TABLE IF NOT EXISTS camaras (
  id          BIGSERIAL PRIMARY KEY,
  nombre      TEXT      NOT NULL,

  url         TEXT      NOT NULL,

  protocolo   TEXT      NOT NULL
                  CHECK (protocolo IN ('rtsp','http','https','file','device')),

  usuario     TEXT,
  password    TEXT,

  lugar_id    BIGINT REFERENCES lugares(id) ON DELETE SET NULL,

  camara_out  BOOLEAN    NOT NULL DEFAULT FALSE,

  habilitada  BOOLEAN    NOT NULL DEFAULT TRUE,
  ultimo_ping TIMESTAMPTZ,
  ultimo_frame TIMESTAMPTZ,

```

```
modelo      TEXT      NOT NULL DEFAULT 'yolo11l.pt',
tracker     TEXT      NOT NULL DEFAULT 'bytetrack.yaml',
img_size    INTEGER   NOT NULL DEFAULT 640,
conf        REAL      NOT NULL DEFAULT 0.45,
stride      INTEGER   NOT NULL DEFAULT 1,
classes     INTEGER[] NOT NULL DEFAULT '{0}',
```

```
roi          JSONB,
line_crossing JSONB,
```

```
credenciales JSONB,
extras        JSONB,
```

```
notas        TEXT,
```

```
created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
-- Evita nombres duplicados dentro del mismo lugar
CONSTRAINT uq_camaras_nombre_lugar UNIQUE (nombre, lugar_id)
);
```

```
CREATE INDEX IF NOT EXISTS idx_camaras_lugar      ON camaras (lugar_id);
CREATE INDEX IF NOT EXISTS idx_camaras_habilitada ON camaras (habilitada);
CREATE INDEX IF NOT EXISTS idx_camaras_protocolo  ON camaras (protocolo);
```

```
CREATE TABLE IF NOT EXISTS metricas (
  id          BIGSERIAL PRIMARY KEY,
  camara_id   BIGINT NOT NULL REFERENCES camaras(id) ON DELETE
  CASCADE,
```

```
ts           TIMESTAMPTZ NOT NULL, -- timestamp que envía el worker (ISO8601)
-- Métricas actuales (las que ya envías)
count        INTEGER   NOT NULL, -- personas detectadas en frame actual
unique_count INTEGER   NOT NULL DEFAULT 0, -- "unicos" vistos desde que
partió
max_count    INTEGER,
min_count    INTEGER,
```

```

roll_avg    DOUBLE PRECISION,    -- promedio móvil ventana
fps         DOUBLE PRECISION,

-- Métricas futuras útiles para line crossing / dirección
in_count    INTEGER,
out_count   INTEGER,

-- Para extensibilidad sin romper esquema
extra       JSONB,

created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

-- Idempotencia por (cámara, segundo) para evitar duplicados de reintentos
ts_sec      TIMESTAMPTZ GENERATED ALWAYS AS (date_trunc('second', ts))
STORED,
CONSTRAINT uq_mtricas_camara_ts UNIQUE (camara_id, ts_sec)
);

CREATE INDEX IF NOT EXISTS idx_mtricas_camara_ts ON metricas (camara_id,
ts DESC);
CREATE INDEX IF NOT EXISTS idx_mtricas_ts      ON metricas (ts DESC);


CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_lugares_updated ON lugares;
CREATE TRIGGER trg_lugares_updated
BEFORE UPDATE ON lugares
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

DROP TRIGGER IF EXISTS trg_camaras_updated ON camaras;
CREATE TRIGGER trg_camaras_updated
BEFORE UPDATE ON camaras
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE OR REPLACE VIEW camaras_estado AS

```

```

SELECT DISTINCT ON (c.id)
  c.id          AS camara_id,
  c.nombre      AS camara,
  c.lugar_id,
  c.habilitada,
  c.camara_out,
  m.ts          AS ultima_medicion,
  m.count,
  m.unique_count,
  m.max_count,
  m.roll_avg,
  m.fps
FROM camaras c
LEFT JOIN metricas m
  ON m.camara_id = c.id
ORDER BY c.id, m.ts DESC;

```

```

DROP TABLE IF EXISTS metricas CASCADE;

```

```

CREATE TABLE metricas (
  id          BIGSERIAL PRIMARY KEY,
  camara_id   BIGINT NOT NULL REFERENCES camaras(id) ON DELETE
  CASCADE,

  ts          TIMESTAMPTZ NOT NULL,          -- timestamp del worker (ISO8601)

  count       INTEGER NOT NULL,
  unique_count INTEGER NOT NULL DEFAULT 0,
  max_count   INTEGER,
  min_count   INTEGER,
  roll_avg    DOUBLE PRECISION,
  fps         DOUBLE PRECISION,
  in_count    INTEGER,
  out_count   INTEGER,

  extra       JSONB,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  ts_sec      TIMESTAMPTZ NOT NULL
);

```

```
-- Índices
CREATE INDEX idx_metricas_camara_ts ON metricas (camara_id, ts DESC);

-- Trigger para rellenar ts_sec truncado al segundo
CREATE OR REPLACE FUNCTION metricas_set_ts_sec()
RETURNS TRIGGER AS $$
BEGIN
    NEW.ts_sec := date_trunc('second', NEW.ts);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_metricas_ts_sec
BEFORE INSERT OR UPDATE OF ts
ON metricas
FOR EACH ROW
EXECUTE FUNCTION metricas_set_ts_sec();

-- Única por (camara, segundo)
CREATE UNIQUE INDEX uq_metricas_camara_tssec ON metricas (camara_id,
ts_sec);
```